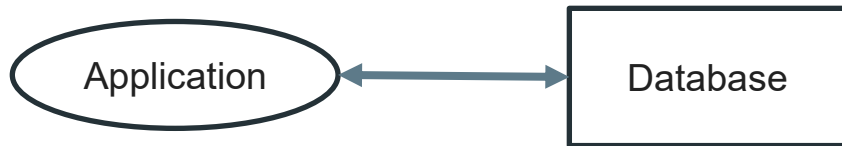


TD7: Ingeniería de Datos

Arquitectura de datos

Arquitecturas de datos

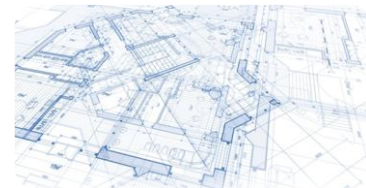
— — —



Cómputo incremental (Fully incremental)

- Se almacena una *única verdad*: la más reciente.
 - Al procesar un nuevo evento (hecho), se actualiza la *verdad* y se descarta la historia.
-
- *Un banco almacena para cada cuenta **únicamente su saldo** (verdad actual)*
 - *Ante una transacción **se actualizan** los saldos de las cuentas involucradas*
 - *La información de la transacción misma **se descarta***

Arquitecturas de datos - fully incremental

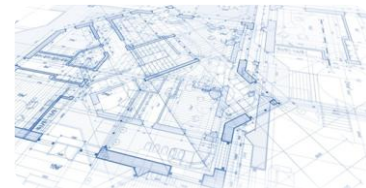


Cuenta	Saldo
pablo	2.000
julia	5.500
tobias	-300



Cuenta	Saldo
pablo	1.900
julia	5.600
tobias	-300

Arquitecturas de datos - fully incremental



— — —
 t^2



Cuenta	Saldo
pablo	1.900
julia	5.600
tobias	-300

t^3



Cuenta	Saldo
pablo	1.900
julia	5.100
tobias	200

La cuestión de la CONSISTENCIA

Modelos de consistencia:
determina la visibilidad y el orden
aparente de las actualizaciones.

Es básicamente un contrato entre
los procesos y el almacén de datos.
Este contrato dice que, si los
procesos aceptan obedecer ciertas
reglas, el almacén promete
funcionar correctamente.

Consistencia fuerte: todas las operaciones de lectura deben devolver datos de la última operación de escritura completada.

Consistencia eventual: las operaciones de lectura verán, conforme pasa el tiempo, las escrituras. En un estado de equilibrio, el estado devolvería el último valor escrito.

A medida que

$t \longrightarrow \infty$

los clientes verán las escrituras

Consistencia no estricta

Read Your Own Writes (RYOW) consistency

El cliente lee sus actualizaciones inmediatamente después de que hayan sido completadas, independientemente si escribe en un server y lee de otro. Las actualizaciones de otros clientes no tienen por qué ser visibles instantáneamente.

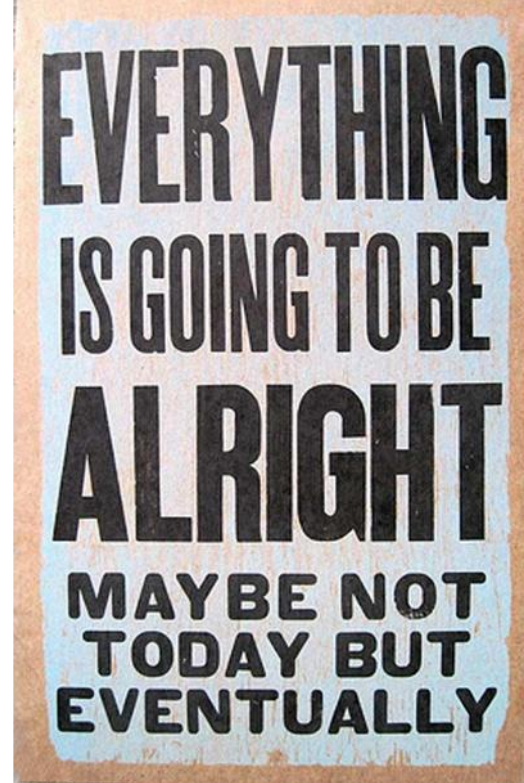
Consistencia no estricta

Consistencia de sesión

Un poco más débil que RYOW. Provee ésta consistencia solo si el cliente está dentro de la misma sesión. Usualmente sobre el mismo server.

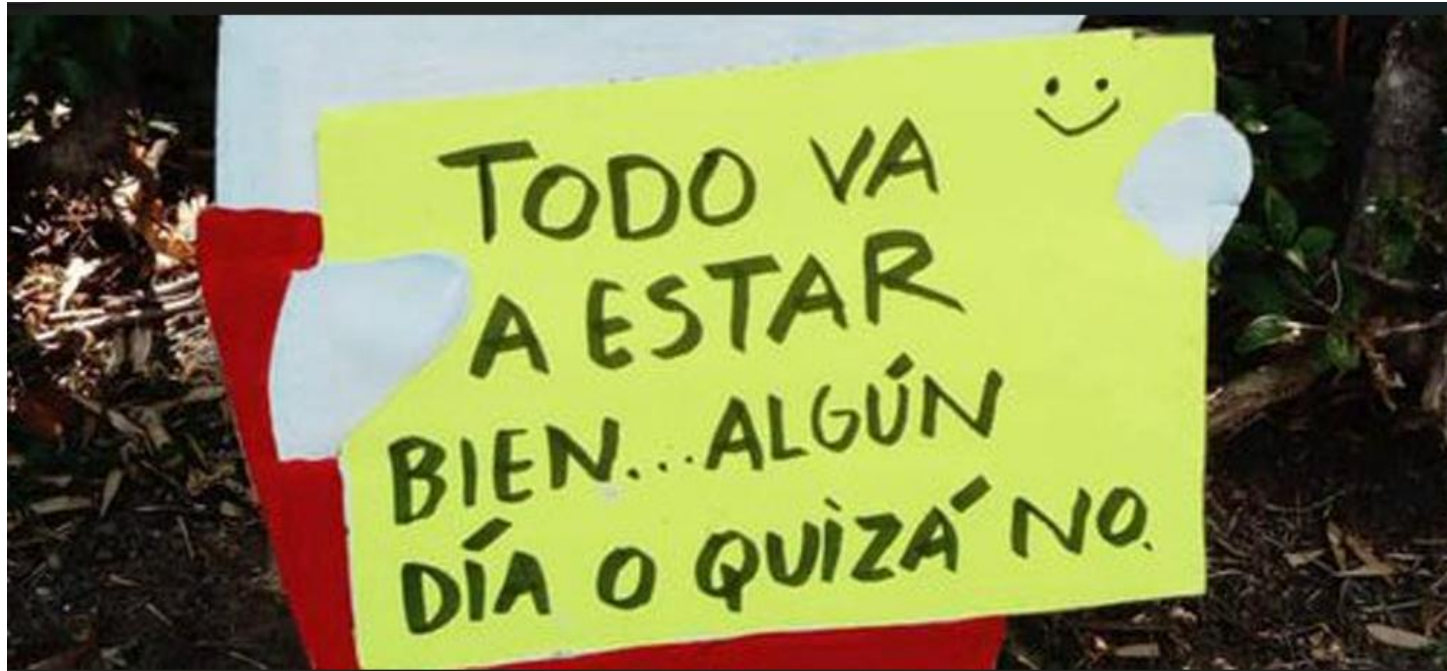
Consistencia no estricta

Consistencia eventual



Consistencia no estricta

Consistencia eventual



Teorema

CAP

Consistency



todos ven los mismos datos al mismo tiempo

Availability



si se puede comunicar con un nodo en el cluster,
entonces se pueden leer y escribir datos

Partition tolerance



el cluster puede sobrevivir a roturas de
comunicación que lo dividan en particiones que
no pueden comunicarse entre ellas

Es imposible para cualquier sistema de computación distribuida proveer las tres propiedades simultáneamente.

Las 8 Falacias de la Computación Distribuida

1. La red es confiable
2. La latencia es cero; la latencia no es problema
3. El ancho de banda es infinito
4. La red es segura
5. La topología no cambia
6. Hay uno y solo un administrador
7. El coste de transporte es cero
8. La red es homogénea

Teorema CAP

Las 8 Falacias de la Computación Distribuida

Consistency

Availability

Partition tolerance

Teorema

CAP

Las 8 Falacias de la Computación Distribuida

¡Hay que tolerar particiones!

CP - Consistency/Partition Tolerance

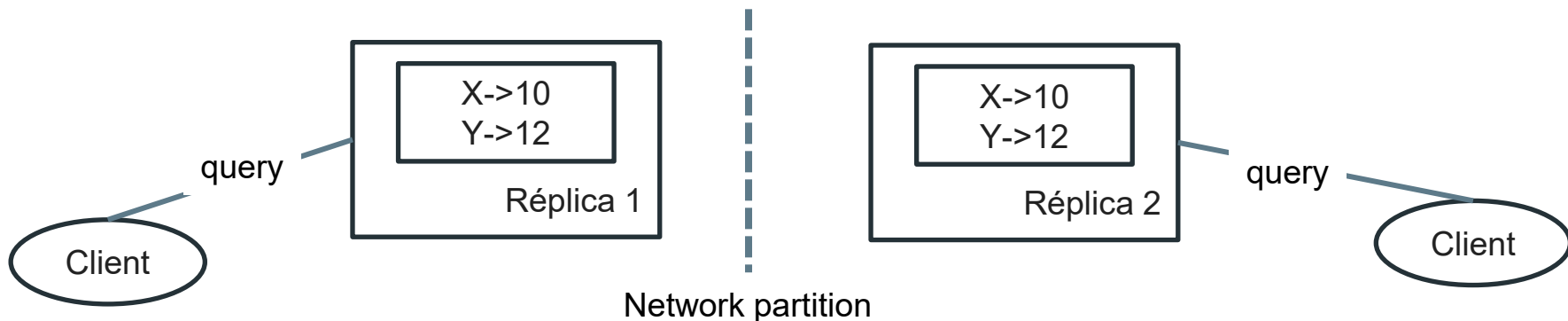
AP - Availability/Partition Tolerance

Arquitecturas de datos - fully incremental



Problemas

- Tratar de hacer Sistemas **altamente disponibles**. Es decir, permitir queries y updates incluso ante la falla de una máquina o de la red.
- Las DBs distribuidas logran gran disponibilidad manteniendo múltiples réplicas de toda la información.



Arquitecturas de datos - fully incremental



Problemas

- No es tolerante a **errores humanos**: si hay un bug en el componente que procesa los datos, no puede corregirse retroactivamente (solucionable operacionalmente)
- No se pueden realizar operaciones sobre **datos históricos** (predicciones)

Arquitecturas de datos - event sourcing



Event sourcing

- Se almacenan solamente todos los **eventos (facts)** desde el inicio de los tiempos.
 - Los eventos son **inmutables**.
 - Una consulta implica **recorrer todos los eventos** relevantes y aplicar los cálculos que correspondan
 - Se pueden consultar *fotos* de **momentos previos** o estimar **momentos futuros**
-
- *Un banco almacena para cada cuenta únicamente su historial de transacciones*
 - *Ante una transacción se almacena el evento*
 - *Al hacerse una consulta de saldo, se recorren todas las transacciones pertinentes y se calcula el saldo actual*

Arquitecturas de datos - event sourcing



— — —



Tiempo	Desde	Hacia	Monto
1704067200	lucas	julia	20.000
1704412800	julia	pablo	2.000
1704603600	pedro	carla	1.500



Tiempo	Desde	Hacia	Monto
1704067200	lucas	julia	20.000
1704412800	julia	pablo	2.000
1704603600	pedro	carla	1.500
1705035600	pablo	julia	100

Arquitecturas de datos - event sourcing



— — —

t^2



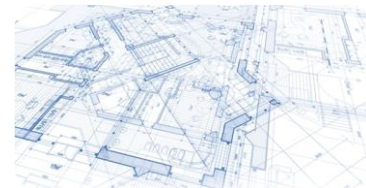
Tiempo	Desde	Hacia	Monto
1704067200	lucas	julia	20.000
1704412800	julia	pablo	2.000
1704603600	pedro	carla	1.500
1705035600	pablo	julia	100

t^3



Tiempo	Desde	Hacia	Monto
1704067200	lucas	julia	20.000
1704412800	julia	pablo	2.000
1704603600	pedro	carla	1.500
1705035600	pablo	julia	100
1705294800	julia	tobias	500

Arquitecturas de datos - event sourcing



— — —

Para calcular el saldo de julia:

- Comenzar en 0
- **+20.000** por transferencia de lucas
- **-2.000** por transferencia a pablo
- [No la afecta la transferencia entre pedro y carla]
- **+100** por transferencia de pablo
- **-500** por transferencia a tobias
- Total = 17.600

Tiempo	Desde	Hacia	Monto
1704067200	lucas	julia	20.000
1704412800	julia	pablo	2.000
1704603600	pedro	carla	1.500
1705035600	pablo	julia	100
1705294800	julia	tobias	500

Arquitecturas de datos - event sourcing

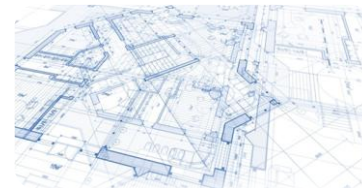


Problemas

- Requiere **mantener** tanto el mecanismo para ingresar eventos como el cómputo para calcular la verdad actual
- Implica mayores necesidades de **almacenamiento**
- Puede traer problemas de escala para calcular verdades actuales cuando **la historia es grande**

Arquitecturas de datos

— — —



Fully incremental	Event sourcing
Implementación intuitiva	Complejidad extra
Bajo costo de almacenamiento	Mayor costo de almacenamiento
Verdad actual no requiere cómputo	Cómputo lineal para obtener valores
Dificultad para recuperarse ante errores	Capacidad de recomputar historia
Sólo verdad actual	Capacidad de ver fotos pasadas o estimar futuro

Cierre



Bibliografía

 **“Big data: Principles and best practices of scalable realtime data systems”**, N. **Marz**, J. Warren, 1st edition, 2015.
 A new paradigm for Big Data: Capítulo 1